Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



Team Details

a. Team name: **Royal Stack**
b. Team leader name: **SATHISHKUMAR S**
c. Problem Statement: Retail & Commerce. One AI commerce agent on one inventory truth for small retailers: it serves shoppers in the aisle and online in their own language, plans the owner's inventory, prevents loss at self-checkout, and keeps in step with the billing software the shop already runs.

Brief about the idea

Royal Stack: one AI commerce agent, one inventory truth.

A Gemini agent that serves the shopper in the aisle and online, runs the owner's inventory, and stays in sync with the billing software the shop already has. The shopper scans a QR at the entrance, talks in any language, follows the map, scans items, pays and leaves through a verified exit. The owner sees forecasts, reorder drafts, demand signals, customers, offers and loss on one dashboard, and can run it all by voice.

In the aisle

The gap: Nobody to ask, no answer in your own language, a queue at the end.

What we built: A live voice assistant that knows every shelf, answers from stock data, lights up the map and lets you scan and pay yourself.

In the back office

The gap: Reorders from memory, demand noticed after the sale is lost, shrink found at stock-take.

What we built: Nightly forecasts, reorder drafts with a reason, demand signals from conversations, a loss view, every change waiting for approval.

Between systems

The gap: Billing software, online store and shoppers each hold a different stock count.

What we built: A connector agent that brings any export, sheet, API or database in, and signed sync that keeps every surface on the same stock.

Everything on these slides is deployed and in use at royastacxk-aibuildercup.web.app, built on Google Cloud with Gemini. Nothing here is planned-only.

Opportunities

- How different is it from any of the other existing ideas?
- How will it be able to solve the problem?
- USP of the proposed solution

How it is different

- Most retail AI is a chatbot on one channel. This is a multi-agent system on Google ADK: coordinators route to specialist agents across three surfaces, in store, online and the owner's desk, all on one stock record.
- Scan-first carts and a rules-based exit check make self-checkout safe for a small shop.
- It plugs into the shop's existing billing software, sheets or database instead of replacing them.

How it solves the problem

- Shoppers ask in any language and get stock, price and aisle from tools, never guessed; out-of-stock items come with alternatives and pack maths.
- Owners get ARIMA forecasts, reorder drafts, demand signals and a loss view, with approval before anything changes.
- The connector agent normalises any source with a preview, so the stock the agent speaks from is the stock the shop bills from.

USP

- "The model talks, code decides": prices, stock, discounts, points and exit verdicts are code inside transactions; Gemini handles language, vision, grounding and proposals.
- 25 languages verified on the live model, Tamil spoken and transcribed in Tamil script.
- A one-person shop registers and is live in a minute; every control is traceable to a test.

List of features offered by the solution

Shopper, in store and online

- Entrance QR, no app install; optional phone sign-in for history, offers and points
- Live speech-to-speech assistant, interruptible, any language, memory across visits
- Text chat with product photos ("what is this?") and product facts with citations
- Floor map that lights up the aisle and bay; aisle QRs update where you stand
- Scan-first cart, offers that apply themselves, points at checkout, sandbox payment
- Signed receipt QR and a verified exit with a bag check

Owner

- Dashboard: today's sales, stock health, decisions waiting
- Inventory with days of cover, BigQuery ML forecasts, reorder drafts with a rationale, approvals
- Demand signals: asked but not carried, asked but out of stock
- Customers in segments, offers (product, category, bill, segment), points ledger with audit
- Loss and shrink view, exit verdict review, kiosk PIN controls
- Owner assistant by text or voice with full, confirm-before-acting access

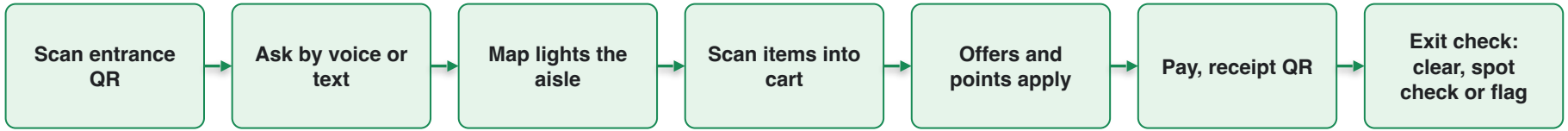
Platform and integration

- Connector agent: files (CSV, Excel, JSON, PDF), Google Sheets, API feeds, read-only Postgres/MySQL, with mapping, preview and confirm
- Signed webhooks in and out, pull endpoint, scheduled feeds, outbox retries
- Multi-floor floor-map editor; chiller and freezer placement by product type
- Self-service registration, roles for owner, staff, shopper and guest
- Echo guard for loudspeaker use; transcripts in the spoken script
- Data export and deletion for shoppers; audit trail for every owner action

Every feature is live today; the shopper side needs no app install, the owner side registers a shop in a minute.

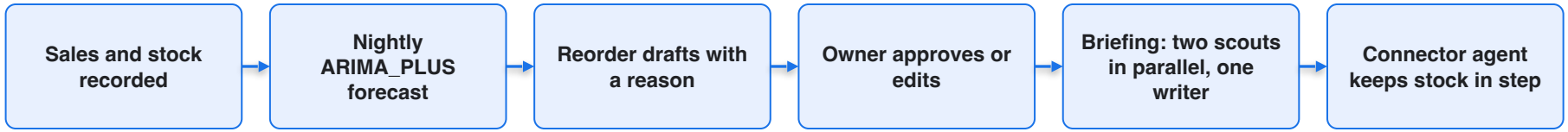
Process flow diagram or Use-case diagram

Shopper journey



A coordinator agent routes each turn to a specialist (catalog, navigation, cart); every answer comes from a tool call on the live catalog, and the screen renders tool results only. Out of stock is said plainly with alternatives; "10 kg" becomes two 5 kg packs.

Owner loop



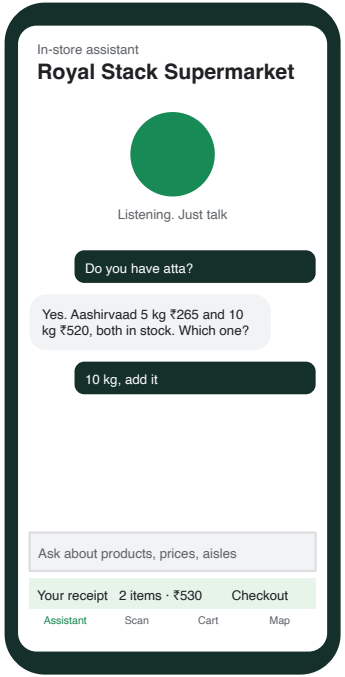
Rules decide at the exit

Receipt token, paid order, item count, line limit, high-value items, random audit, then a Gemini bag photo that only moves confidence. Staff review flags with a kiosk PIN.

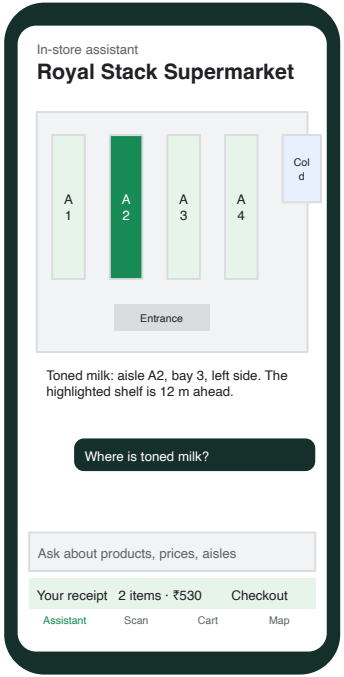
Bring in the shop's own data

Upload an export, paste a Sheet or API link, or connect a read-only database. The agent proposes the column mapping, previews adds, updates and restocks, writes after confirmation, and pulls again every 15 minutes.

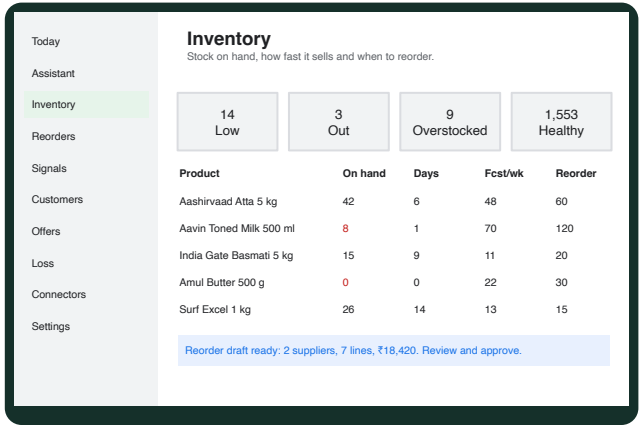
Wireframes/Mock diagrams of the proposed solution (optional)



Assistant: talk or type, receipt always visible



Map lights the aisle and bay



Owner dashboard: inventory with forecasts, days of cover and reorder drafts

Mock-ups drawn to the live layouts. Shoppers see the receipt grow as they scan; the exit kiosk and the owner pages share the same catalog.



Live: Tamil conversation on a phone

Architecture diagram of the proposed solution

Clients

Shopper PWA on the phone (in store and online) · Owner dashboard on laptop or phone · Exit kiosk on a tablet. Static Next.js app on Firebase Hosting; Firebase Auth: Google and email for owners, phone OTP for shoppers who opt in.

Cloud Run: FastAPI + Google ADK agent teams

- Coordinator agents route each turn to specialists (shopper: catalog, navigation, cart; owner: inventory planner, customers and offers, data connector, store ops); the owner briefing runs two scouts in parallel then a writer; research and mapping agents are called as tools
- Gemini Live bridge over websockets: echo detection, transcript language hints, memory of record in Firestore
- Product facts with Google Search grounding; connector readers for files, Sheets, APIs and databases
- Signed sync in and out (HMAC, replay-safe), rate limits, audit on every change

Vertex AI · Gemini

- gemini-3.5-flash: tools, vision, mapping, facts
- gemini-live-2.5-flash: speech to speech
- gemini-embedding-001: multilingual search
- Any of 25 languages, verified

Firestore

Inventory truth, orders, points ledger, memory, audit. Deny-all client rules.

BigQuery ML

Sales history; ARIMA_PLUS forecasts per product, nightly.

Cloud Scheduler

Forecasts, customer jobs, sync and connectors every 15 min.

Secret Manager

QR signing key, job key, per-store payment, sync and connector secrets.

Outside the platform

Shop's billing software, Google Sheets, POS APIs, Postgres/MySQL (through the connector agent and signed sync) · Open Food Facts for barcodes · Google Search grounding for product facts · Razorpay (sandbox) for payment · GitHub Actions CI with the Firestore emulator, Cloud Build source deploys

Technologies to be used in the solution

Google Cloud and AI

- Vertex AI Gemini 3.5 Flash (tools, vision, grounding)
- Gemini Live 2.5 Flash (speech to speech)
- gemini-embedding-001 (search)
- Google ADK 2.12: coordinator, specialist, parallel and sequential agents
- Cloud Run, Cloud Build, Cloud Scheduler
- Firestore, BigQuery ML (ARIMA_PLUS)
- Secret Manager

Firebase and web

- Firebase Hosting (static PWA, no-cache pages)
- Firebase Auth: Google, email, phone OTP
- Next.js 16, TypeScript, Tailwind v4, zustand
- Web Audio + AudioWorklet for live voice
- Barcode and QR scanning in the browser
- Recharts, qrcode.react

Backend and data

- Python 3.13, FastAPI, google-genai SDK
- SQLAlchemy, psycopg, PyMySQL, openpyxl, httpx
- HMAC-signed sync contract, outbox with retries
- Kaggle BigBasket and Myntra catalogues, Open Food Facts
- pytest on the Firestore emulator (325 tests), vitest (30), GitHub Actions CI
- Design notes per module in docs/lld, control map in docs/security.md

Why these: Gemini for language, vision, grounding and proposals; Firestore transactions for anything that counts or pays; BigQuery ML for forecasts without an ML pipeline; Cloud Run so one shop costs a warm instance, not a server.

Estimated implementation cost (optional)

Indicative monthly cost for one shop at pilot scale (about 300 visits a day, one warm API instance, asia-south1). List prices change; usage-based lines scale with conversations.

Component	What drives it	Indicative cost per month
Cloud Run API	One warm instance (1 vCPU, 1 GiB), scales to three under load	₹1,200 to ₹2,500
Gemini usage	Voice: about ₹5 per visit of 8 turns. Text: under ₹0.50 per question. Forecast rationale and mapping: a few paise each	₹4,000 to ₹12,000 depending on voice share
Firestore	Catalog, carts, orders, memory; mostly inside the free tier at this scale	₹0 to ₹500
BigQuery ML	Nightly ARIMA_PLUS retraining per store, small tables	₹0 to ₹300
Firebase Hosting and Auth	Static hosting, Google and email sign-in free; phone OTP per SMS beyond the free allowance	₹0 to ₹500
Scheduler, Secret Manager, Build	Three jobs, a handful of secrets, source deploys	under ₹200
Total	Platform fixed cost plus usage; no per-seat licence, no hardware beyond a phone and an optional tablet	about ₹6,000 to ₹16,000

Snapshots of the prototype



Shopper assistant in Tamil, headphones detected, live voice

What the live product shows today

- Shopper (phone): entrance QR, live voice with interruption and echo guard, text chat with photos, product facts with sources, map, scan-first cart, offers, points, checkout, receipt QR
- Owner (laptop or phone): today's sales, inventory with forecasts and reorder drafts, demand signals, customers, offers, loss view, floor-map editor, connector agent, owner assistant by voice
- Exit kiosk (tablet): receipt scan, bag check, clear / spot check / flag with a staff PIN

See it yourself

- royastacxk-aibuildercup.web.app: open the shopper app on a phone; the demo store has 1,579 products
- /owner/: register a shop of your own in a minute with Google or email; import an export through Connectors
- /kiosk/: the exit kiosk flow (store id and PIN from the owner's Loss tab)

Demo data

- Royal Stack Supermarket: 1,579 grocery products, six aisles plus chiller and freezer
- Captain Jack Mart: 300 BigBasket products; Kingsman Textiles: 1,046 Myntra products
- Synthetic sales history behind the forecasts; Open Food Facts for real barcodes

Prototype Performance report/Benchmarking

Measured on the deployed service on 18 Oct 2026, from a laptop in India, medians of three runs.

Measure	Result
Catalog read (store, 1,579 products)	154 ms
Text question with tool calls, first visible response	2.0 to 2.8 s
Text question, full answer streamed (search, stock, cart add)	7 to 11 s
Live voice: first spoken audio after the question	3.3 s
Live voice: a 21-second spoken answer fully delivered	6.2 s (streams while it speaks)
Catalog refresh across API instances after a stock change	within 2 s (version counter)
Languages accepted by the live model for transcription hints	25 verified; Tamil speech shown in Tamil script
Loudspeaker echo loop (assistant interrupting itself)	Eliminated: browser echo guard plus server text and timing detection, field tested
Automated tests	325 backend on the Firestore emulator, 30 frontend; live-model scenarios for voice, Tamil, owner language switch and grounded facts
Demo catalogue	2,925 products across three stores (grocery 1,579; BigBasket 300; Myntra textiles 1,046) with embeddings and forecasts

Additional Details/Future Development (if any)

Next 30 days

- Named adapters on the connector agent: Shopify, Zoho Inventory, Tally Prime exports
- Owner screen for the audit trail and connector runs
- GST invoices and returns against a receipt
- 90-day retention job for verification records and transcripts

Next quarter

- MCP server so any external shopping assistant can search, reserve and buy from a Royal Stack store
- Supplier portal: purchase orders sent and acknowledged
- Shelf-camera stock checks feeding the same forecast
- Store-to-store transfers for owners with several shops

Pilots and the business

- Self-service registration is open today; a shop is live in a minute with its own export
- Pricing idea: a flat platform fee per shop plus usage, no hardware
- Target: neighbourhood supermarkets and textile shops in Tamil Nadu first, then any JAPAC market the model speaks
- Measure: lost-sale signals recovered, shrink reduced, queue time removed

The platform is self-service today: a new shop registers, imports its export through the connector agent, prints its QR codes and is live.

Provide links to your:

- GitHub Public Repository
- Demo Video Link (3 Minutes)
- Final Product Link

GitHub public repository: https://github.com/DSsathish/royalstack_aibuilderhackathon

Demo video (3 minutes): Demo video: link to be added after recording

Final product link: https://royastacxk-aibuildercup.web.app

Owner dashboard at /owner/ (self-service registration), exit kiosk at /kiosk/. Demo store: Royal Stack Supermarket.

API and docs: https://retail-api-881917398995.asia-south1.run.app/docs

Design notes in docs/lld, control map in docs/security.md, demo script in docs/demo-script.md.

Trust, safety and privacy by design

The model talks, code decides

- Coordinators only route; specialists only act through tools; stock, prices and locations come from tool results, never from memory
- Discounts, points and exit verdicts are rules and arithmetic inside Firestore transactions
- Every stock change, order or offer waits for the owner's confirmation; all of it audited

Guardrails on the open web

- Product facts: product topics only, citations required, 200-character questions, a budget per visit, nothing about the shopper sent
- Connector agent: https only, public addresses only, read-only database access, credentials in Secret Manager
- Prompt-injection stance: web pages and files are data, never instructions

Privacy and access

- Shoppers see only the last four digits of their number; they can export or delete everything
- Spoken turns stay on the phone; the exit bag photo keeps only a hash, a confidence and item names
- Roles for owner, staff, shopper and guest; signed entrance and receipt tokens; kiosk PIN lockout; rate limits

docs/security.md in the repository maps every control to its code path and its test, and lists the known gaps.

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

